

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/254045262>

On hardware implementation of radix 3 and radix 5 FFT kernels for LTE systems

Article · November 2011

DOI: 10.1109/NORCHP.2011.6126703

CITATIONS

23

READS

5,519

2 authors, including:



Peter Nilsson

Lund University

105 PUBLICATIONS 2,398 CITATIONS

SEE PROFILE

On Hardware Implementation of Radix 3 and Radix 5 FFT Kernels for LTE systems

Johan Löfgren and Peter Nilsson

Dept. of Electrical and Information Technology, Box 118, Lund University, Sweden

Email: {Johan.Lofgren, Peter.Nilsson}@eit.lth.se

Abstract—This paper treats the hardware architecture and implementation of mixed radix FFTs with cores of radix 3 and radix 5 in addition to the standard radix 2 core. The implementation flow graphs of the higher radix cores are presented together with a description of how these cores affect a pipelined FFT implementation. It is shown that the mixed radix FFT is more expensive than the radix 2 implementation - a mixed radix FFT of 1200 points require 36 real multipliers in a pipelined implementation whereas a 2048 radix 2 FFT needs 30 real multipliers. However, half of the multipliers in the mixed radix case can be constant. Therefore it is still feasible to use the mixed radix FFT if an algorithm calls for it.

I. INTRODUCTION

The 3GPP Long Term Evolution (LTE) mobile phone system is currently being deployed world wide [1]. This 4G system allows for much higher data rates than its predecessors. The target rate is 100 MBits/s or above in good conditions. Future updates, called LTE advanced, allows for even higher data rates, up to 1 GBit/s. In order to reach these high data rates, the system must be optimized and used efficiently.

The standard is based on orthogonal frequency division multiplexing (OFDM). The transmitted data is spread over a number of subcarriers on different frequencies. To spread the data the Fast Fourier Transform (FFT) is employed. For the most part the FFT size can be a standard 2^n point FFT. However there are situations where other FFT sizes are beneficial. The minimum frequency allocation unit - the resource block (RB) - contains 12 subcarriers, which cannot be factored in twos only [1]. Further, the maximum number of resource blocks allocated in one band is 100, which also cannot be factored in twos only.

Certain channel estimation algorithms that want to use the available subcarriers, require FFTs of 1200 or 2400 points [2]. However, there are no implementations of FFTs of these, needed, sizes described in the literature. There are reports on radix 3 FFT kernel architectures [3] and implementations [4]. Other kernel sizes, including radix 5, have been described for computing, though not specifically aimed for hardware implementation [5]. In this paper the hardware architecture of the radix 3 and radix 5 FFT kernels are being investigated. The derivation of hardware implementable blocks is done and a feasible implementation of a 1200 point FFT is described.

The rest of the paper is organized as follows. In Section II the different radix 3 and radix 5 kernels are presented. Thereafter, in Section III the hardware implementable pipelined FFT is generalized to handle also radix 3 and radix 5. Section IV

then describes the full 1200 point FFT implementation, whereafter the paper is concluded in Section V.

II. FAST FOURIER TRANSFORM

The Discrete Fourier Transform (DFT) is defined as

$$X(k) = \sum_{n=0}^{N-1} x(n)W_N^{kn} \quad (1)$$

where $x(n)$ is a sequence of N input data and W_N^{kn} is the so called twiddle factor [6].

Calculating the DFT directly, using (1) requires a large number of operations $\mathcal{O}(N^2)$. However, there are symmetries in the calculations that allows for large reductions in complexity. One of the most widely used methods to leverage this symmetry and achieve a Fast Fourier Transform is the method developed by Cooley-Tukey [7]. They describe a way to divide the DFT into two smaller DFTs so that $N = N_1N_2$, i.e., the product of the the sizes of the new DFTs is equal to the size of the original DFT. This can be recursively continued down to the prime factors of the size of the original DFT. Using this method the complexity can be reduced to $\mathcal{O}(N \log N)$.

A. Radix 2

In hardware implementations small prime factors are used, since they lead to the best reductions. Therefore systems are most often designed for FFTs of size $N = 2^m$ where m is an integer. The DFT is thus broken down into m DFTs of size 2. This split operation is shown in (2) and (3).

$$\begin{aligned} X(k) &= \sum_{n=0}^{N-1} x(n)W_N^{kn} \\ &= \sum_{n=0}^{N/2-1} x(n)W_N^{kn} + \sum_{n=N/2}^{N-1} x(n)W_N^{kn} \\ &= \sum_{n=0}^{N/2-1} x_1(n)W_N^{kn} + \underbrace{W_N^{kN/2}}_{(-1)^k} \sum_{n=0}^{N/2-1} x_2(n)W_N^{kn} \end{aligned} \quad (2)$$

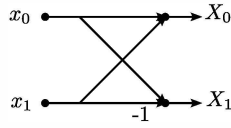


Fig. 1. Radix 2 Flow Graph

TABLE I
CALCULATION STAGES IN A RADIX 2 UNIT

Stage 1
$B_0 = x_0 + x_1$
$B_1 = x_0 - x_1$

$$\begin{aligned}
X(2k) &= \sum_{n=0}^{N/2-1} (x_1(n) + x_2(n)) W_{N/2}^{kn} \\
&= \text{DFT}_{N/2}(x_1(n) + x_2(n)) \\
&= \text{DFT}_{N/2}(B_0) \\
X(2k+1) &= \sum_{n=0}^{N/2-1} ((x_1(n) - x_2(n)) W_N^n) W_{N/2}^{kn} \\
&= \text{DFT}_{N/2}((x_1(n) - x_2(n)) W_N^n) \\
&= \text{DFT}_{N/2}(B_1 W_N^n)
\end{aligned} \tag{3}$$

where $B_0 = x_0 + x_1$ and $B_1 = x_0 - x_1$ with the time index, n , dropped. Thus, the initial DFT is split into two new DFTs, with arguments B_0 and $B_1 W_N^n$.

The calculation of B_0 and B_1 can be represented in a flow graph as shown in Fig. 1 [6]. This graph is often referred to as a butterfly graph due to its shape.

As can be seen in the equations, the output of the lower path of the butterfly, B_1 , needs to be multiplied by a twiddle factor, W_N^n . This is the only multiplication needed in the radix 2 FFT.

It is also possible to represent the flow graph in tabular form, where the output is shown as functions of the inputs. In some cases this representation may need one or more intermediate results, or stages. However, in the case of a radix 2 representation this is not needed, as illustrated in Table I, where a single stage is shown.

B. Radix 3

Similar to the radix 2 split it is possible to split a DFT into radix 3 units, if the original DFT is of a size that can be factored down to one or more threes, i.e., $N = 3^m$. In the radix 3 case the calculation is split into three different DFTs, instead of two. The different equations are shown in (4).

$$\begin{aligned}
X(3k) &= \text{DFT}_{N/3}(B_0) \\
X(3k+1) &= \text{DFT}_{N/3}(B_1 W_N^n) \\
X(3k+2) &= \text{DFT}_{N/3}(B_2 W_N^{2n})
\end{aligned} \tag{4}$$

where $B_0 = x_0 + x_1 + x_2$, $B_1 = x_0 + x_1 W_3^1 + x_2 W_3^2$, and $B_2 = x_0 + x_1 W_3^2 + x_2 W_3^1$.

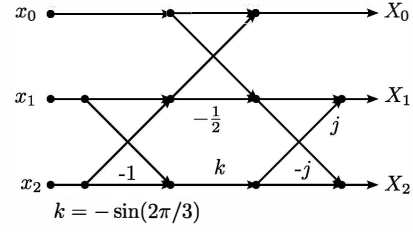


Fig. 2. Radix 3 Flow Graph

TABLE II
CALCULATION STAGES IN A RADIX 3 UNIT

Stage 1	Stage 2	Stage 3
$a_0 = x_0$	$b_0 = a_0 + a_1$	$B_0 = d_0$
$a_1 = x_1 + x_2$	$b_1 = a_0 - \frac{1}{2}a_1$	$B_1 = d_1 + jd_2$
$a_2 = x_1 - x_2$	$b_2 = ka_2$	$B_2 = d_1 - jd_2$

To further simplify the calculations it is noted that

$$\begin{aligned}
\Re(W_3^1) &= \Re(W_3^2) = -\frac{1}{2} \text{ and} \\
\Im(W_3^1) &= -\Im(W_3^2) = -\sin\left(\frac{2\pi}{3}\right)
\end{aligned}$$

where $\Re(\cdot)$ is the real and $\Im(\cdot)$ is the imaginary part of the number.

This allows a flow graph as shown in Fig. 2. In this flow graph two internal multiplications, in addition to -1 and j , are shown. However, one of them is a trivial multiplication with $\frac{1}{2}$ and will therefore not add to the hardware complexity. It can be noted that it is possible to make radix 3 unit without internal multiplications [3]. This requires a non-trivial change of basis for the inputs and outputs and is therefore only beneficial in case of a DFT with several radix 3 units. In addition to the internal multiplications, two of the three outputs also need to be multiplied with twiddle factors, as can be seen in (4).

If this flow graph is presented in tabular form, a number of intermediate stages are needed. This is shown in Table II.

C. Radix 5

If the DFT instead can be factored into a multiple of fives, i.e., the size is $N = 5^n$, radix 5 units are needed. The derivation of this is handled the same way as for radix 2 and 3 units. Since these units have five inputs and five outputs, the basic equation is split into five different equations as shown in (5).

$$\begin{aligned}
X(5k) &= \text{DFT}_{N/5}(B_0) \\
X(5k+1) &= \text{DFT}_{N/5}(B_1 W_N^n) \\
X(5k+2) &= \text{DFT}_{N/5}(B_2 W_N^{2n}) \\
X(5k+3) &= \text{DFT}_{N/5}(B_3 W_N^{3n}) \\
X(5k+4) &= \text{DFT}_{N/5}(B_4 W_N^{4n})
\end{aligned} \tag{5}$$

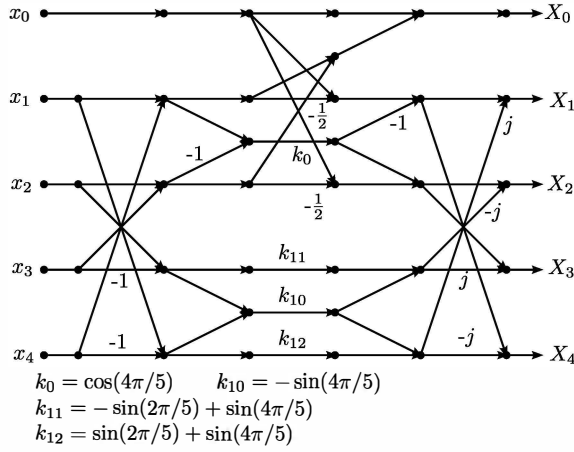


Fig. 3. Radix 5 Flow Graph

with

$$\begin{aligned}
 B_0 &= x_0 + x_1 + x_2 + x_3 + x_4 \\
 B_1 &= x_0 + x_1 W_5^1 + x_2 W_5^2 + x_3 W_5^3 + x_4 W_5^4 \\
 B_2 &= x_0 + x_1 W_5^2 + x_2 W_5^4 + x_3 W_5^1 + x_4 W_5^3 \\
 B_3 &= x_0 + x_1 W_5^3 + x_2 W_5^1 + x_3 W_5^4 + x_4 W_5^2 \\
 B_4 &= x_0 + x_1 W_5^4 + x_2 W_5^3 + x_3 W_5^2 + x_4 W_5^1.
 \end{aligned}$$

To further simplify the calculations it is noted that

$$\begin{aligned}
 \Re(W_5^1) &= \Re(W_5^4) = \cos\left(\frac{2\pi}{5}\right), \\
 \Re(W_5^2) &= \Re(W_5^3) = \cos\left(\frac{4\pi}{5}\right), \\
 \Re(W_5^1) + \Re(W_5^2) &= -\frac{1}{2}, \\
 \Im(W_5^1) &= -\Im(W_5^4) = -\sin\left(\frac{2\pi}{5}\right), \text{ and} \\
 \Im(W_5^2) &= -\Im(W_5^3) = -\sin\left(\frac{4\pi}{5}\right)
 \end{aligned}$$

where, as above, $\Re(\cdot)$ is the real and $\Im(\cdot)$ is the imaginary part of the number.

This allows for calculating the outputs using the flow graph presented in Fig. 3. As can be seen, this radix 5 unit has four internal multiplications in addition to the two trivial divisions by 2. Also four of the five outputs needs to be twiddled.

Table III shows the flow graph broken down into different stages. Since the flow graph for the radix 5 is larger, more intermediate steps are needed. The intermediate steps also require more values than the five inputs.

D. Mixed Radix

It is also possible to create FFTs of different sizes by combining units of different radices, to get an FFT of size $N = 2^n 3^m 5^p$. The units can be combined in any order. It is thus possible to create a 1200 point FFT, eg., by $N_{1200} = 2^4 3^1 5^2$, $N_{1200} = 5^2 3^1 2^4$, or $N_{1200} = 2^2 5^1 3^1 2^2 5^1$.

Even though all combinations lead to the same FFT, there is a difference in the output order. It is well known that the *decimation in frequency* radix 2 FFT leads to a bit-reversed output order. This concept can be generalized when other radices are used, into digit-reversed output, where each digit

TABLE III
CALCULATION STAGES IN A RADIX 5 UNIT

Stage 1	Stage 2	Stage 3
$a_0 = x_0$	$b_0 = a_0$	$c_0 = b_0$
$a_1 = x_1 + x_4$	$b_1 = a_1$	$c_1 = b_0 - \frac{1}{2}b_1$
$a_2 = x_2 + x_3$	$b_2 = a_2$	$c_2 = b_0 - \frac{1}{2}b_2$
$a_3 = x_2 - x_3$	$b_3 = a_3$	$c_3 = k_{11}b_3$
$a_4 = x_1 - x_4$	$b_4 = a_4$	$c_4 = k_{12}b_4$
	$b_5 = a_1 - a_2$	$c_5 = k_0b_5$
	$b_6 = a_3 + a_4$	$c_6 = k_{10}b_6$
		$c_7 = b_1 + b_2$

Stage 4	Stage 5
$d_0 = c_0 + c_7$	$B_0 = d_0$
$d_1 = c_1 - c_5$	$B_1 = d_0 + jd_4$
$d_2 = c_2 + c_5$	$B_2 = d_2 + jd_3$
$d_3 = c_3 + c_6$	$B_3 = d_2 - jd_3$
$d_4 = c_4 + c_6$	$B_4 = d_0 - jd_4$

TABLE IV
OUTPUT ORDER IN A 6 POINT FFT

Input Order	Output $N_{61} = 3^1 2^1$	Output $N_{62} = 2^1 3^1$
0 ₁₀	0 ₂₀₃ = 0 ₁₀	0 ₃₀₂ = 0 ₁₀
1 ₁₀	1 ₂₀₃ = 3 ₁₀	1 ₃₀₂ = 2 ₁₀
2 ₁₀	0 ₂₁₃ = 1 ₁₀	2 ₃₀₂ = 4 ₁₀
3 ₁₀	1 ₂₁₃ = 4 ₁₀	0 ₃₁₂ = 1 ₁₀
4 ₁₀	0 ₂₂₃ = 2 ₁₀	1 ₃₁₂ = 3 ₁₀
5 ₁₀	1 ₂₂₃ = 5 ₁₀	2 ₃₁₂ = 5 ₁₀

will be in the same base as the radix of the FFT unit. To clarify the concept consider the two possibilities of a small FFT $N_{61} = 3^1 2^1$ and $N_{62} = 2^1 3^1$. Their respective output orders are shown in Table IV. As can be seen, the most significant digit is increased first, thus leading to the digit-reversed order.

III. PIPELINED FFT ARCHITECTURE

It is possible to implement the FFT flow graph directly in hardware. However, for large FFTs it may be infeasible since the area usage would be too large. This means that the FFT implementation is often folded, so that throughput is traded for a smaller area. A common folded structure is the pipeline structure [6] as shown in Fig. 4 for a radix 2 FFT.

This architecture has a single input and a single output and generates one sample per iteration. To ensure the correct order of the input data each pipeline stage needs a First-In-First-Out queue (FIFO) to store input data. Each stage of the architecture works in two passes. In the first pass data is read directly into

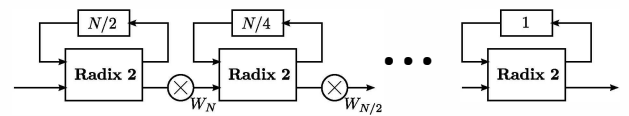


Fig. 4. Pipelined Radix 2 FFT

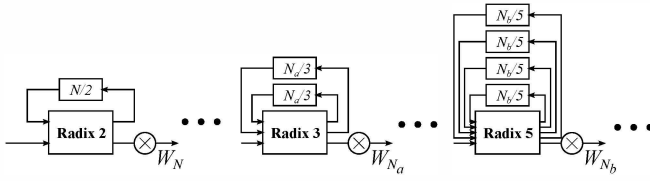


Fig. 5. Pipelined Mixed Radix FFT

the FIFO at the same time as the current content of the FIFO is streamed to the next stage. Thereafter, in the second pass, the input data is combined with the data in the FIFO and sent to the butterfly. One of the outputs in this pass is sent to the next stage and the other is stored in the FIFO to be output in the next run of the first pass. In the structure shown here, implementing a *decimation in frequency* FFT, the initial FIFOs need to be longer than the ones in the end, to ensure the correct data handling. The first FIFO needs to contain half of the input data and is thus of length $N/2$, whereas the last unit only requires a single data to be stored. Also, for each stage fewer and fewer twiddling factors need to be used. In the final stage all twiddling factors are unity and the multiplier unit can be removed.

It is possible to generalize the pipeline structure in an FFT based on other radices. Since each flow graph requires more inputs, some minor modifications have to be made. The largest is that the number of FIFOs for each stage will be increased. In a radix 3 unit, two FIFOs are needed and in a radix 5 four FIFOs are needed as shown in Fig. 5.

As can be seen, more storage is required for the higher radix flow graphs. The radix 3 unit needs to store $\frac{2}{3}$ of the input data and the radix 5 needs to store $\frac{4}{5}$. Hence, it is beneficial to put the higher radix units at the end of the pipeline.

IV. MIXED RADIX COMPARISON

In order to give an idea of the size and cost of a mixed radix FFT an example of a 1200 point FFT is examined and compared with a 1024 and a 2048 point radix 2 FFT. The comparison is done both for a fully unrolled implementation, where all multiplications are constant and with the pipelined structure where the multipliers have variable twiddle factors.

The 1200 point FFT is laid out with the four radix 2 units first, followed by a radix 3 unit and then ended in two radix 5 units as $N_{1200} = 2^4 3^1 5^2$ for a total of 7 units. The 1024 point FFT needs 10 radix 2 units for $N_{1024} = 2^{10}$ whereas the 2048 point FFT needs one extra radix 2 unit for a total of 11 units, $N_{2048} = 2^{11}$.

The total number of multipliers for the different FFT sizes are shown in Table V. One complex multiplication is translated into three real multiplications to give a reasonable comparison. Trivial multiplications, ie., multiplication with $0 + 1j$ has not been counted.

It can be seen that the number of multiplications is higher in the 1200 point FFT than in the pure radix 2 versions.

TABLE V
CALCULATION COST COMPARISON

Unrolled	1200 Points	1024 Points	2048 Points
# Real Mults	16958	10758	24582
# Mults per point	14.1	10.5	12.0
Pipelined	1200 Points	1024 Points	2048 Points
# Stages	7	10	11
# FIFO Registers	1199	1023	2047
# Real Mults	36*	27	30

*18 variable and 18 constant multipliers

However, in the pipelined architecture this will not necessarily correspond to a larger implementation since half of the multipliers will be constant multipliers, inside the flow graphs, and will therefore be smaller than the variable twiddle factor multipliers. Also, the number of registers needed increase linearly with the number of inputs, in both the radix 2 and the mixed radix cases.

V. CONCLUSION

The presentation in this paper shows that it is feasible to use other radices in an FFT than the standard radix 2. It is shown that in the fully unrolled FFT the cost is rather high. The number of multiplications per input data for a 1200 point FFT is 14.1. This should be compared with 10.5 for the 1024 point and 12.0 for the 2048 point radix 2 FFTs respectively. For the pipelined implementation of the FFT the cost goes down, however. This is due to the fact that in a pipelined architecture, the multipliers are often not needed in the radix 2 based FFT. The higher utilization in the mixed radix FFT shows that it is possible to use this configuration when called for by an algorithm.

REFERENCES

- [1] E. Dahlman, S. Parkvall, J. Skold, and P. Beming, *3G Evolution, Second Edition: HSPA and LTE for Mobile Broadband*, 2nd ed. Academic Press, 2008.
- [2] J. Lofgren, O. Edfors, and P. Nilsson, "Improved matching pursuit algorithm and architecture for lte channel estimation," in *Circuits and Systems (ISCAS), 2011 IEEE International Symposium on*, may 2011, pp. 466–469.
- [3] E. Dubois and A. Venetsanopoulos, "A new algorithm for the radix-3 fft," *Acoustics, Speech and Signal Processing, IEEE Transactions on*, vol. 26, no. 3, pp. 222–225, jun 1978.
- [4] S.-Y. Peng, K.-T. Shr, C.-M. Chen, and Y.-H. Huang, "Energy-efficient 128~2048/1536-point fft processor with resource block mapping for 3gpp-lte system," in *Green Circuits and Systems (ICGCS), 2010 International Conference on*, june 2010, pp. 14–17.
- [5] S. Goedecker, "Fast radix 2, 3, 4, and 5 kernels for fast fourier transformations on computers with overlapping multiply-add instructions," *SIAM J. Sci. Comput.*, vol. 18, pp. 1605–1611, November 1997.
- [6] S. He and M. Torkelson, "A new approach to pipeline fft processor," in *Parallel Processing Symposium, 1996., Proceedings of IPPS '96, The 10th International*, apr 1996, pp. 766–770.
- [7] J. Cooley and J. Tukey, "An algorithm for the machine calculation of complex fourier series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.